

Capstone Project Briefs

12 Projects · Student Edition

This document contains the official project briefs for the capstone component of the Automation with Generative AI course. Each brief defines exactly what you are expected to build, what must be included, what your README must document, and what tech stack to use.

Projects are grouped into three tracks based on the core technology: AI model focused, automation focused, and combined. You choose one project. Two students may choose the same project — your implementation is individual and will be compared.

How you will be reviewed

Your submission will be reviewed by Claude AI against this brief document, and then by a QAE reviewer who will interact with your live deployed URL. Claude checks whether your implementation covers what is listed under Must Include. The QAE reviewer checks whether it actually works and whether the output is useful and complete.

What to submit

Public GitHub repository URL with your complete code

A README.md that covers every point listed in the README Requirements section of your chosen brief

Live deployed URL (Render recommended)

Project 01

Prompt Battle Arena

"Which model actually wins for your use case?"

WHAT YOU ARE BUILDING

Build a Streamlit app that lets a user define any task and a custom scoring rubric. The app fires the same prompt at Claude, GPT-4o, and Gemini simultaneously using their APIs. A fourth AI call scores each response against the user-defined rubric. Results are displayed as a side-by-side scored comparison with reasoning for each score.

MUST INCLUDE — ALL OF THESE ARE REQUIRED

- User can define their own task prompt and scoring rubric (at least 3 criteria)
- All three models (Claude, GPT-4o, Gemini) called in parallel or sequence
- A fourth AI call acts as judge and scores each response per rubric criterion
- Scores displayed visually — table or bar chart showing winner per criterion
- Overall winner declared with reasoning
- Export full battle report (prompts, responses, scores, reasoning) as PDF
- Error handling if any one model API call fails — show partial results, not crash
- API keys loaded from environment variables, not hardcoded

README MUST DOCUMENT

- Which models are supported and how to configure API keys for each
- How the scoring rubric input works and what format it expects
- How the judge AI call is structured (which model, what prompt)
- Known limitations — latency, cost per battle, token limits

RECOMMENDED STACK

[Claude API](#) · [OpenAI API](#) · [Gemini API](#) · [Streamlit](#) · [python-dotenv](#)

Why this project stands out: You built a framework to objectively compare AI models. That tells me you understand prompt engineering AND system design.

Project 02

Contract Risk Scanner

"Upload any contract, get a risk briefing in 60 seconds."

WHAT YOU ARE BUILDING

Build a Streamlit app where a user uploads a contract PDF or pastes contract text. The AI reads the full contract and produces a structured risk report — identifying risky clauses, missing standard protections, unusual terms, and liability exposure. Each finding is rated by severity.

MUST INCLUDE — ALL OF THESE ARE REQUIRED

- Accept PDF upload or plain text paste as input
- AI identifies at minimum: risky clauses, missing standard protections, unusual terms, liability exposure

- Each finding rated as High / Medium / Low severity with explanation
- Results displayed as a structured report in the app — not just raw AI text
- Download full risk report as formatted PDF with severity colour coding
- Handles contracts up to 10 pages without crashing
- Graceful message if document is too short or not a contract
- API key from environment variable

README MUST DOCUMENT

- Which AI model is used and why (Claude recommended for long context)
- How the risk categories are defined in the prompt
- How severity ratings are determined
- Limitations — not legal advice, model may miss complex clauses

RECOMMENDED STACK

Claude API · Streamlit · pypdf · python-dotenv

Why this project stands out: This is exactly the kind of internal tool legal and procurement teams are asking for. You identified a real pain point.

Project 03

Cold Outreach Personaliser

"Paste a LinkedIn profile, get a personalised email that does not sound AI-generated."

WHAT YOU ARE BUILDING

Build a Streamlit app where a user pastes a prospect profile (LinkedIn about section, bio, or job description), selects tone (formal / casual / direct) and goal (intro call / partnership / job application). The AI generates a personalised outreach email with subject line, body, and a suggested follow-up message. Batch mode lets users upload a CSV of prospects and download personalised emails for all of them.

MUST INCLUDE — ALL OF THESE ARE REQUIRED

- Single mode: paste one profile, pick tone and goal, get personalised email + subject + follow-up
- Batch mode: upload CSV with name and profile columns, generate email for each row
- Tone selector with at least 3 options affecting the output measurably
- Output shows email in copy-ready format in the app
- Batch download: CSV with original data plus generated email and subject columns
- Single export: download individual email as text file
- Handles empty or very short profiles gracefully
- API key from environment variable

README MUST DOCUMENT

- How to format the input CSV for batch mode (column names, example row)
- How tone selection is implemented in the prompt
- Cost estimate per email and per batch of 50
- Limitations — output quality depends on how detailed the input profile is

RECOMMENDED STACK

Claude API · Streamlit · python-dotenv

Why this project stands out: Batch personalisation with tone control is a tool sales teams would actually pay for. And you built it.

Project 04

Study Guide Generator

"Upload any lecture PDF, get a complete study pack."

WHAT YOU ARE BUILDING

Build a Streamlit app where a student uploads a lecture PDF or pastes syllabus content. The AI generates a complete study pack: concise summary notes, 20 practice MCQs with answers and explanations, 5 short-answer questions with model answers, a key terms glossary, and a suggested study order. Student picks difficulty level (beginner / intermediate / advanced).

MUST INCLUDE — ALL OF THESE ARE REQUIRED

- Accept PDF upload or text paste
- Difficulty level selector that measurably changes output complexity
- Generate all five components: summary, MCQs, short answer, glossary, study order
- MCQs must have 4 options each with the correct answer and a brief explanation
- Download complete study pack as formatted PDF
- Export MCQs only as CSV (question, option A/B/C/D, correct answer) for flashcard import
- Handles PDFs up to 15 pages
- API key from environment variable

README MUST DOCUMENT

- How difficulty level affects the prompt and output
- CSV format for MCQ export and which flashcard tools it is compatible with
- Token usage estimate per document page
- Limitations — works best with factual academic content, not pure maths

RECOMMENDED STACK

Claude API · Streamlit · pypdf · python-dotenv

Why this project stands out: You built something students would actually use and pay for. That tells me you thought about the end user, not just the tech.

Project 05

Code Reviewer And Explainer

"Paste code, get a senior developer review without the ego."

WHAT YOU ARE BUILDING

Build a Streamlit app where a user pastes code in any language and selects review depth (quick scan / deep review). The AI produces four outputs: a line-by-line explanation in plain English for beginners, a list of bugs and security vulnerabilities with line references, a refactored version with improvements, and a complexity and readability assessment. Output shows original and improved code side by side.

MUST INCLUDE — ALL OF THESE ARE REQUIRED

- Language auto-detected from the code (no manual selection required)
- Two review depth modes with meaningfully different output detail
- Four distinct output sections: explanation, bugs, refactored code, assessment
- Bug list includes line number references where possible
- Side-by-side display of original vs refactored code in the app
- Export full review as markdown file
- Handles code up to 200 lines
- Graceful message for non-code input
- API key from environment variable

README MUST DOCUMENT

- How language detection works in the prompt
- Difference between quick scan and deep review modes
- Limitations — AI may miss subtle logic bugs, output is advisory not authoritative
- Supported languages tested

RECOMMENDED STACK

[Claude API](#) · [Streamlit](#) · [python-dotenv](#)

Why this project stands out: A code review tool that also explains for beginners — junior developers would use this daily. Smart target audience.

Track 2 — Automation Focused · n8n as the Core

Project 06

Job Application Autopilot

"Submit once, get a personalised cover letter, a fit score, and a follow-up reminder automatically."

WHAT YOU ARE BUILDING

Build an n8n workflow triggered by a Google Form submission. The applicant pastes a job description and uploads their CV text. n8n sends both to the AI which scores CV fit against the JD (0-100 with reasoning) and generates a personalised cover letter. n8n emails the cover letter to the applicant, logs everything to a Google Sheet, and creates a calendar reminder 7 days later to follow up.

MUST INCLUDE — ALL OF THESE ARE REQUIRED

- Google Form collects: name, email, job description (text), CV content (text)
- n8n workflow triggers automatically on form submission — no manual steps
- AI produces: fit score 0-100 with breakdown by criteria, and personalised cover letter
- Cover letter emailed to applicant automatically with fit score summary
- Google Sheet logs: name, email, JD snippet, fit score, timestamp, cover letter link or text
- Calendar reminder created 7 days after submission for follow-up
- If AI call fails, error logged to Sheet and applicant emailed a fallback message
- API key stored in n8n credentials, not exposed in workflow nodes

README MUST DOCUMENT

- n8n workflow structure — node by node explanation with screenshot
- How fit scoring criteria are defined in the prompt
- Google Sheet column structure
- How to set up the calendar integration and which calendar service is used

RECOMMENDED STACK

[n8n](#) · [Claude API](#) · [Google Forms](#) · [Google Sheets](#) · [Gmail](#) · [Google Calendar](#)

Why this project stands out: You automated the actual job hunting workflow and thought about the follow-up. That is end-to-end thinking.

Project 07

Social Media Content Pipeline

"One idea becomes a week of content across three platforms automatically."

WHAT YOU ARE BUILDING

Build an n8n workflow where a user submits a content idea via Google Form (idea, target audience, brand tone). n8n sends it to the AI which generates a LinkedIn post, a Twitter/X thread (5-7 tweets), and an Instagram caption — all from the same idea but formatted correctly for each platform. Drafts are posted to a Slack channel for human review. When a reviewer reacts with a specific emoji, the content is logged to a Google Sheet publishing calendar with a scheduled date.

MUST INCLUDE — ALL OF THESE ARE REQUIRED

- Google Form collects: content idea, target audience, brand tone, preferred publish date
- n8n triggers automatically on form submission
- AI generates three distinct outputs: LinkedIn post, Twitter thread, Instagram caption
- Each output formatted correctly for its platform (character limits, hashtags, thread numbering)
- All three drafts posted to a Slack channel in a structured message
- Emoji reaction on the Slack message triggers approval and logs to Sheets calendar
- Google Sheet has columns: platform, content, publish date, status, approved by
- If AI call fails, error message sent to Slack with the original brief

README MUST DOCUMENT

- n8n workflow diagram showing all nodes and branches
- Which Slack emoji triggers approval and how to configure it
- Google Sheet column structure and how to use it as a publishing calendar
- How brand tone affects the prompt and output

RECOMMENDED STACK

[n8n](#) · [Claude API](#) · [Google Forms](#) · [Google Sheets](#) · [Slack](#)

Why this project stands out: A content pipeline with human approval built in — that is production thinking, not just automation.

Project 08

Customer Feedback Intelligence

"Every support email becomes a product insight automatically."

WHAT YOU ARE BUILDING

Build an n8n workflow triggered by incoming email. The AI classifies each email as bug report, feature request, complaint, or praise — then extracts the product mentioned, severity level, and a key quote. n8n routes the email summary to the correct Slack channel by category and logs a structured row to Google Sheets. A weekly n8n cron job aggregates the week's data and sends a trend summary report by email to the product team.

MUST INCLUDE — ALL OF THESE ARE REQUIRED

- Email trigger in n8n — works with Gmail or a webhook from an email service
- AI classifies into exactly four categories with confidence indicator
- Extraction includes: product name, severity (high/medium/low), key quote under 30 words
- Routing sends to different Slack channels per category
- Google Sheet logs: timestamp, category, product, severity, quote, original email subject
- Weekly cron (every Monday 9am) generates trend summary: top issues, category counts, most mentioned product
- Trend summary emailed to a configured product team email address
- If classification confidence is low, routed to an uncategorised Slack channel for manual review

README MUST DOCUMENT

- n8n workflow structure with separate diagrams for main flow and cron flow
- How to configure the email trigger for Gmail
- Slack channel naming convention and how to set up routing
- How the weekly trend summary is generated and what it includes

RECOMMENDED STACK

n8n · **Claude API** · **Gmail** · **Google Sheets** · **Slack**

Why this project stands out: You connected support to product intelligence and built in a weekly digest. That is a real business workflow.

Project 09

Interview Question Generator

"Paste a job description, get a complete interview kit ready in 2 minutes."

WHAT YOU ARE BUILDING

Build an n8n workflow triggered by a Google Form where a hiring manager submits a job title, full JD, and seniority level. The AI generates 20 interview questions across technical, behavioural, and situational categories — with a scoring rubric for each question explaining what a strong answer looks like. n8n emails the complete kit as a formatted PDF to the hiring manager and logs to Google Sheets.

MUST INCLUDE — ALL OF THESE ARE REQUIRED

- Google Form collects: job title, job description, seniority level (junior/mid/senior)
- n8n triggers automatically on form submission

- AI generates exactly 20 questions: at least 7 technical, 7 behavioural, 6 situational
- Each question includes: category tag, what to listen for, what a strong answer looks like
- Complete kit formatted as PDF and emailed to the hiring manager
- Google Sheet logs: job title, seniority, timestamp, number of questions generated, email sent status
- Seniority level measurably changes question difficulty and technical depth
- If AI call fails, fallback email sent with error notification

README MUST DOCUMENT

- n8n workflow node diagram
- How seniority level affects the prompt
- PDF format and structure of the interview kit
- Google Sheet column structure

RECOMMENDED STACK

[n8n](#) · [Claude API](#) · [Google Forms](#) · [Google Sheets](#) · [Gmail](#)

Why this project stands out: HR teams waste hours on this manually. You automated it with a thoughtful output structure and a clear use case.

Track 3 — Combined · n8n + Python + Streamlit

Project 10

Research Assistant With Memory

"Drop URLs, ask questions across all of them like they are one document."

WHAT YOU ARE BUILDING

Build a system where an n8n webhook receives URLs, scrapes the content, chunks and embeds it into ChromaDB. A Streamlit app lets the user ask questions across all ingested sources. Claude retrieves relevant chunks and answers with citations showing which URL each fact came from. Users can add more URLs at any time without losing previous ones. Full Q&A; session can be exported as PDF.

MUST INCLUDE — ALL OF THESE ARE REQUIRED

- n8n webhook accepts URL input and triggers scraping and embedding pipeline
- Content chunked and embedded using OpenAI embeddings API into ChromaDB
- Streamlit app shows list of ingested sources with timestamps
- User can ask questions in natural language
- Claude answer must cite which source URL each fact comes from
- If answer is not found in sources, AI says so explicitly rather than hallucinating
- New URLs can be added without wiping existing knowledge base
- Export full Q&A; session with citations as PDF
- API keys from environment variables

README MUST DOCUMENT

- Architecture diagram showing n8n, ChromaDB, and Streamlit components
- How to run n8n and Streamlit together locally
- Chunking strategy used and why

- How citation tracking works in the retrieval prompt
- Limitations — scraping may fail for JavaScript-heavy sites

RECOMMENDED STACK

[n8n](#) · [Claude API](#) · [OpenAI Embeddings API](#) · [ChromaDB](#) · [Streamlit](#) · [python-dotenv](#)

Why this project stands out: A RAG system with persistent ingestion and source citations — that is production architecture. Most graduates cannot build this.

Project 11

Meeting Intelligence Dashboard

"Paste a meeting transcript, never lose an action item again."

WHAT YOU ARE BUILDING

Build a system where an n8n webhook receives meeting transcripts. The AI extracts action items with owner names and deadlines, key decisions, open questions, and a 3-line summary. A Streamlit dashboard shows all meetings in a timeline view. Users filter by owner name to see their action items across all meetings. Overdue items are highlighted automatically. Export action items as CSV.

MUST INCLUDE — ALL OF THESE ARE REQUIRED

- n8n webhook receives transcript text and triggers AI extraction
- AI extracts: action items (with owner + deadline), decisions, open questions, 3-line summary
- All extractions stored persistently (JSON file or SQLite) — not lost on app restart
- Streamlit dashboard shows all meetings in reverse chronological order
- Filter by owner name shows only that person's action items across all meetings
- Overdue items (deadline passed) highlighted in red automatically
- Export all action items across all meetings as CSV with columns: meeting date, item, owner, deadline, status
- API key from environment variable

README MUST DOCUMENT

- Architecture diagram showing n8n webhook and Streamlit app
- How to send a transcript to the n8n webhook (curl example or form)
- Storage approach — where data is persisted between sessions
- How overdue detection works
- Limitations — owner name extraction depends on transcript format

RECOMMENDED STACK

[n8n](#) · [Claude API](#) · [Streamlit](#) · [python-dotenv](#)

Why this project stands out: A multi-meeting dashboard with owner filtering — you thought about the actual user workflow, not just the demo.

Project 12

Competitor Intelligence Tracker

"Track what your competitors are saying and doing automatically, every week."

WHAT YOU ARE BUILDING

Build a system where an n8n cron job runs weekly, fetches content from configured competitor blog URLs and social pages, and sends it to the AI which summarises new content, extracts product announcements, pricing changes, and positioning shifts. A Streamlit dashboard shows a competitor timeline with category tags. A Monday morning email digest is sent automatically. Users can export the weekly report as PDF.

MUST INCLUDE — ALL OF THESE ARE REQUIRED

- n8n cron triggers every Monday at a configured time
- At least 3 competitor URLs configured and fetched each run
- AI extracts per source: new content summary, product announcements, pricing mentions, positioning language
- Only new content since last run is processed — deduplication required
- Streamlit dashboard shows timeline per competitor with category tags
- Filter by competitor or category (announcement / pricing / positioning / general)
- Email digest sent automatically every Monday with that week's highlights
- Export current week report as PDF
- API key from environment variable

README MUST DOCUMENT

- How to configure competitor URLs in the system
- Deduplication approach — how the system knows what is new
- n8n cron schedule configuration
- Dashboard filter options and how to use them
- Limitations — scraping may fail for paywalled or JS-heavy sites

RECOMMENDED STACK

[n8n](#) · [Gemini API](#) · [Streamlit](#) · [python-dotenv](#) · [Gmail](#)

Why this project stands out: Automated competitive intelligence with a scheduled digest. Marketing and product teams would pay for this immediately.